Lab 3: Parser

Continue developing the language you designed in Labs 1 and 2, using Lox and the textbook's implementation as references. Implement a parser that converts scanned tokens into an abstract syntax tree (AST), and use your AST printer to display the result. Explain your design choices and differences from Lox in your report.

Reuse your scanner, AST classes, and AST printer from previous labs, and include everything needed to run your code.

Tasks

You will submit both your **code** and a **report** for this lab. Partial credit will be awarded for all tasks.

Report quality and submission correctness together account for at most **10% of the total score**, included within the task weights below.

- **Parser implementation (40%):**
  Implement a `Parser` class or equivalent structure based on your language's expression grammar. It must construct an AST using your AST classes and correctly handle operator precedence, associativity, and grouping.

  Support the literal types and expression operators defined in your grammar. In Lox, these include binary `+`, `-`, `/`, and `*`; unary `-` and `!`; equality operators `==` and `!=`; comparison operators `>=`, `<=`, `<`, and `>`; and grouping with parentheses. Adapt these to your language's design and support any additional expression operators you defined. Logical `and` and `or` are not required for this lab.

- **Scanner, parser, and AST printer integration (30%):**
  Connect your scanner, parser, and AST printer in the main program so that it:

  1. Accepts an expression from a source file or interactive input.
  2. Scans the input into tokens and parses those tokens into an AST.
  3. Prints the corresponding tree using your AST printer.

  Both source-file and interactive modes must work. Parse one expression per source file or interactive input, and report unexpected trailing tokens instead of silently ignoring them.

- **Error handling (30%):**
  Detect and report syntax errors, including unmatched parentheses, missing operands, and unexpected tokens in expressions. Each diagnostic must identify the problem and location. Handle invalid input without an unhandled exception from the host language, and keep interactive mode usable after an error. Do not print a successful AST for input that contains lexical or syntax errors.

Testing and Report

Write your own test cases covering **every supported expression form, literal type, and operator**, including precedence, associativity, nested grouping, and invalid input. Include at least one example for each supported syntax error. Test both source-file and interactive modes, including recovery after errors.

Place your report under `report/` in your repository. Your report must include:

- Your expression grammar, its precedence and associativity rules, and design choices relative to Lox. Include any grammar changes made since Lab 2.
- Dependency/setup instructions and commands to run test cases.
- For each test case: its purpose, source input, expected AST output or errors, actual program output, and whether the result matches expectations. Output may be included as text or readable screenshots. Briefly explain the cause of each syntax error.
- For one valid expression: a hand-drawn or diagrammed syntax tree that strictly follows your grammar, the program's printed AST, and a short explanation of how the two correspond.
- Any known limitations or failing tests.

Commit the test inputs and any test scripts under `test/lab3/` so that your results can be reproduced.

Optional Extra Credit

You may apply for extra credit if you believe you put extra effort into a feature beyond the basic parser and integration requirements. Examples may include the C-style comma operator, the C-style conditional(ternary) operator `?:`, and detecting the error of missing left operand in a binary expression.

In your report under `report/`, identify the feature, explain the additional effort and its value, and provide test cases and output demonstrating it. Extra credit is subject to instructor evaluation and is capped at **10% of the lab's base grade (10 percentage points)** across all extra-credit features.

Submission Instructions

Submit a **link to the exact commit in your GitHub repository** on Blackboard, using the format `https://github.com/<owner>/<repo>/commit/<commit-hash>`.

The submitted commit must include:

- All implementation files in `src/`, including the parser and reused scanner, AST classes, and AST printer.
- All test inputs and test scripts for this lab in `test/lab3/`.
- The Lab 3 report in `report/`, including the expression grammar, concrete syntax tree, setup/run instructions, test results, and any extra-credit request. Include any image files needed to view the report.

Use the following repository structure (the report filename below is an example):

```
repository/
├── src/                # Implementation files
├── test/
│   └── lab3/           # Lab 3 test inputs and test scripts
└── report/
    └── lab3.md         # Lab 3 report, including setup and run
instructions
```

Ensure that the instructor can access the repository and reproduce your results from the submitted commit.

Late Penalty

You should submit your lab on time. Being late for one lab could affect your time to complete subsequent labs. All labs are due at 11:59 pm (Eastern) of the specified due date, and there is **a 10% penalty each day for up to 50%**. After that, you get zero.

AI Policy

Using AI help for the labs is ALLOWED but discouraged. But you must quote the code from AI help.

Quote it like this:

// --- start AI code ---

// --- end AI code ---

Using AI generated code without quoting is considered academic misconduct and will result in 0 for the lab. Mistakes in the code from AI help will result in doubled point deductions according to the rubrics.

Reuse of code from previous labs is ALLOWED and encouraged.